



SUPPORT DE COURS COMPLET

Algorithmique et Programmation en C

15h CM · 15h TP · 30h

Dr. Zakaria Sawadogo

École Polytechnique de Ouagadougou



Sommaire

Syllabus

1. Chapitre 1 — Introduction à l'algorithmique et au langage C
2. Chapitre 2 — Variables, types, opérateurs et entrées-sorties
3. Chapitre 3 — Structures de contrôle
4. Chapitre 4 — Fonctions et portée des variables
5. Chapitre 5 — Tableaux et chaînes de caractères
6. Chapitre 6 — Pointeurs et gestion mémoire
7. Chapitre 7 — Structures (`struct`) et fichiers

Algorithmique et Programmation en C

Volume horaire : 15h CM · 15h TP (30h)

Objectifs pédagogiques

À l'issue du cours, l'étudiant est capable de :

- concevoir un algorithme structuré pour résoudre un problème donné, avant toute écriture de code ;
- écrire, compiler et déboguer des programmes en langage C ;
- manipuler les structures de données fondamentales (tableaux, chaînes, structures) et la mémoire (pointeurs, allocation dynamique) ;
- lire et modifier du code C existant, compétence indispensable pour l'analyse de vulnérabilités, l'exploitation logicielle et la rétro-ingénierie abordées plus loin dans le programme (Sécurité des applications, Cryptanalyse).

Prérequis

Aucun prérequis en programmation n'est exigé ; des bases de logique mathématique (algèbre de Boole, notions d'arithmétique) sont utiles.

Plan de séances

Cours magistraux (15h)

Séance	Chapitre	Durée
1	Introduction à l'algorithmique et au langage C	2h
2	Variables, types, opérateurs et entrées-sorties	2h
3	Structures de contrôle	2h
4	Fonctions et portée des variables	2h
5	Tableaux et chaînes de caractères	3h
6	Pointeurs et gestion mémoire	2h
7	Structures (struct) et fichiers	2h

Travaux pratiques (15h)

Séance	Sujet	Durée
1	Prise en main : compilation et premiers programmes	2h
2	Structures de contrôle	2h
3	Fonctions	2h
4	Tableaux	2h
5	Chaînes de caractères	2h
6	Pointeurs et allocation dynamique	3h
7	Mini-projet : gestionnaire de contacts	2h

Évaluation

- Contrôle continu (exercices notés en TP) : 40 %
- Mini-projet final (TP7) : 20 %
- Examen final sur table (algorithmique + lecture/écriture de code C) : 40 %

Environnement technique

- Compilateur : GCC (`gcc -Wall -Wextra -std=c11`)
- Éditeur recommandé : VS Code, ou tout éditeur avec coloration syntaxique C
- Système : Linux/macOS/WSL recommandé pour la cohérence des TP

Bibliographie

- B. W. Kernighan, D. M. Ritchie, *The C Programming Language*, 2nd ed.
- C. Delannoy, *Programmer en langage C*, Eyrolles.
- Documentation : cppreference.com/w/c

Chapitre 1 — Introduction à l'algorithmique et au langage C

1. Qu'est-ce qu'un algorithme ?

Un algorithme est une suite finie et non ambiguë d'instructions permettant de résoudre un problème ou d'accomplir une tâche à partir de données d'entrée, en produisant un résultat en un temps fini. Trois propriétés le caractérisent :

- **Finitude** : l'algorithme se termine après un nombre fini d'étapes, quelles que soient les données d'entrée (dans leur domaine de validité).
- **Déterminisme** : chaque étape est définie sans ambiguïté ; à chaque instant, on sait exactement quelle instruction exécuter ensuite.
- **Effectivité** : chaque étape est suffisamment simple et concrète pour être réellement réalisable (par une machine ou, historiquement, par un humain suivant une procédure).

On distingue l'algorithme (la méthode, indépendante de tout langage) du **programme** (sa traduction dans un langage de programmation précis) et de l'**exécution** (le déroulement effectif du programme sur une machine, avec des données concrètes). Un même algorithme peut être implémenté dans des dizaines de langages différents ; c'est pourquoi il est fondamental d'apprendre à concevoir un algorithme *avant* de penser à sa syntaxe C.

Avant d'écrire du code, on décrit donc un algorithme en **pseudo-code**, une notation semi-formelle indépendante de tout langage, qui privilégie la clarté sur la rigueur syntaxique :

```
ALGORITHME Maximum
ENTRÉE : a, b (deux entiers)
SORTIE : le plus grand des deux
DÉBUT
    SI a > b ALORS
        RETOURNER a
    SINON
        RETOURNER b
    FIN SI
FIN
```

Un second exemple : tester la parité d'un entier

```

ALGORITHME EstPair
ENTRÉE : n (un entier)
SORTIE : VRAI si n est pair, FAUX sinon
DÉBUT
    SI (n MOD 2) = 0 ALORS
        RETOURNER VRAI
    SINON
        RETOURNER FAUX
    FIN SI
FIN

```

Correction d'un algorithme : une première intuition

Concevoir un algorithme ne suffit pas : il faut aussi être capable d'argumenter qu'il fait bien ce qu'on attend de lui, pour toutes les entrées valides — et pas seulement pour les quelques cas que l'on a testés « à la main ». Deux notions, formalisées plus tard dans le cursus (analyse d'algorithmes), sont utiles dès maintenant :

- la **précondition** : ce que l'on suppose vrai sur les données d'entrée avant l'exécution (ex. « `b` est différent de zéro » pour une division) ;
- la **postcondition** : ce que l'algorithme garantit être vrai en sortie, si la précondition était satisfaite.

Cette habitude de raisonner en préconditions/postconditions, encore artisanale à ce stade, est la même qui structure plus tard l'analyse de sécurité d'une fonction : une vulnérabilité est très souvent la conséquence d'une précondition supposée par le développeur mais jamais vérifiée dans le code (par exemple, supposer qu'une chaîne de caractères fournie par l'utilisateur ne dépassera jamais une certaine taille).

2. Pourquoi le langage C ?

Le C a été conçu par Dennis Ritchie chez Bell Labs au début des années 1970, en s'appuyant sur le langage B de Ken Thompson, lui-même dérivé du BCPL. Il a servi dès l'origine à réécrire le système d'exploitation Unix, ce qui explique sa proximité durable avec les systèmes d'exploitation et les architectures matérielles. Le langage a ensuite été normalisé par l'ANSI puis l'ISO, avec plusieurs révisions du standard : C89/C90, C99, C11, C17, et plus récemment C23. Dans ce cours, on s'appuie sur un sous-ensemble stable et largement supporté du langage (proche de C11), compatible avec la plupart des compilateurs modernes (GCC, Clang).

Le C reste incontournable, en particulier en sécurité informatique, pour plusieurs raisons :

- il offre un accès direct à la mémoire (arithmétique de pointeurs, allocation manuelle), ce qui en fait le meilleur terrain pédagogique pour comprendre en profondeur les vulnérabilités mémoire (*buffer overflow*, *use-after-free*, *double free*, etc.) plutôt que de les subir comme des boîtes noires ;
- une grande partie des systèmes d'exploitation (noyaux Linux, Windows, BSD), des interpréteurs d'autres langages, des bases de données et des outils bas niveau (compilateurs, pilotes,

systèmes embarqués) sont écrits en C ou s'appuient dessus ;

- comprendre le C facilite la lecture de code décompilé ou désassemblé lors d'une analyse de vulnérabilités, d'une rétro-ingénierie de logiciel malveillant, ou d'une cryptanalyse appliquée ;
- de nombreuses bibliothèques cryptographiques de référence (OpenSSL, libsodium, etc.) sont écrites en C, précisément pour des raisons de performance et de contrôle fin de la mémoire.

Ce que le C ne fait pas à votre place

Contrairement à des langages comme Python, Java ou Rust, le C ne comporte ni ramasse-miettes (*garbage collector*), ni vérification automatique des bornes de tableaux, ni système de types empêchant les conversions dangereuses. Cette absence de filet de sécurité est délibérée : elle donne un contrôle total sur la mémoire et les performances, au prix d'une responsabilité entière laissée au programmeur. C'est précisément cette caractéristique qui fait du C un langage à la fois puissant et propice aux vulnérabilités s'il est mal maîtrisé — d'où l'importance, dans ce cours, d'insister systématiquement sur les pièges et les bonnes pratiques associés à chaque construction du langage.

3. De l'algorithme au programme exécutable

La chaîne de compilation C comporte plusieurs étapes distinctes, généralement invisibles pour le débutant car enchaînées automatiquement par le compilateur :

1. **Préprocesseur** : traite les directives commençant par `#` (`#include`, `#define`, `#ifdef`, ...) *avant* toute analyse du langage C proprement dit. Il s'agit d'une simple substitution textuelle : `#include` copie littéralement le contenu d'un fichier d'en-tête, `#define` remplace un identifiant par un texte.
2. **Compilation** : traduit le code C (après préproression) en assembleur, puis génère un fichier objet (`.o`) contenant du code machine et des symboles non encore résolus.
3. **Édition de liens (linking)** : assemble un ou plusieurs fichiers objets avec les bibliothèques nécessaires (statiques `.a` ou dynamiques `.so/.dll`) pour produire un exécutable final, en résolvant les références entre fonctions/variables définies dans des fichiers différents.

```
gcc -Wall -Wextra -std=c11 -o programme programme.c
./programme
echo $?          # affiche le code de retour du programme (0 = succès)
```

Les avertissements du compilateur comme première ligne de défense

Les options `-Wall` et `-Wextra` activent un grand nombre d'avertissements (variable non utilisée, comparaison suspecte, type incompatible, etc.). Il est vivement recommandé d'aller plus loin en développement :

```
gcc -Wall -Wextra -Werror -std=c11 -fsanitize=address,undefined -g -o programme program
```

- `-Werror` transforme chaque avertissement en erreur bloquante : on ne peut pas « oublier » un avertissement.

- `-fsanitize=address` (AddressSanitizer) instrumente le programme pour détecter à l'exécution les débordements de tampon, les *use-after-free*, etc.
- `-fsanitize=undefined` détecte les comportements indéfinis (division par zéro, débordement d'entier signé, déréférencement de pointeur invalide...).
- `-g` conserve les informations de débogage, utiles avec un débogueur comme `gdb`.

Cette discipline — compiler avec un maximum d'avertissements et d'instrumentation dès le développement — est directement transposable au monde professionnel : la majorité des vulnérabilités mémoire en C auraient pu être détectées bien avant la mise en production, simplement en activant ces options. Des outils d'analyse statique complémentaires (`cppcheck`, `clang-tidy`, `clang --analyze`) et dynamiques (`valgrind`) seront réutilisés au fil des travaux pratiques.

4. Premier programme

```
#include <stdio.h>

int main(void) {
    printf("Bonjour, monde !\n");
    return 0;
}
```

- `#include <stdio.h>` : inclut la bibliothèque d'entrées-sorties standard, nécessaire pour utiliser `printf`.
- `int main(void)` : point d'entrée du programme. Le type `int` en retour signifie que la fonction renvoie un entier au système d'exploitation ; `void` entre parenthèses indique explicitement qu'elle ne prend aucun paramètre (une signature `main()` sans `void` est tolérée mais moins précise).
- `printf` : affiche du texte formaté sur la sortie standard ; `\n` est un caractère d'échappement représentant un saut de ligne.
- `return 0;` : termine `main` en renvoyant le code 0, convention universelle signifiant « succès ». Toute autre valeur (souvent entre 1 et 255) signale une erreur au processus appelant (par exemple un script shell qui teste `$?`).

Un deuxième exemple, avec variables et calcul

```
#include <stdio.h>

int main(void) {
    int cote = 5;
    int perimetre = 4 * cote;

    printf("Le perimetre d'un carre de cote %d est %d.\n", cote, perimetre);
    return 0;
}
```

Ce court exemple illustre déjà l'essentiel du flux d'un programme C : déclarer des données, effectuer un calcul, produire un résultat observable. Les chapitres suivants détaillent chacun de ces aspects (types,

structures de contrôle, fonctions).

5. Structure générale d'un programme C

```
directives de préprocesseur      (#include, #define, ...)
déclarations globales           (constantes, prototypes de fonctions)
int main(void) {
    déclarations locales
    instructions
    return code;
}
définitions des autres fonctions
```

Un fichier source C respecte généralement cet ordre, même si le langage autorise une certaine souplesse. Le prototype d'une fonction (sa signature, sans son corps) doit être connu du compilateur *avant* tout appel à cette fonction ; c'est pourquoi les prototypes sont placés en haut du fichier, ou dans un fichier d'en-tête (.h) inclus via `#include`.

Conventions de nommage et lisibilité

Le C n'impose presque aucune convention de nommage, mais en adopter une de façon cohérente améliore fortement la lisibilité et la maintenabilité du code — deux qualités indissociables d'un code sûr, car un code difficile à lire est un code difficile à auditer :

- noms de variables et de fonctions en minuscules, mots séparés par `_` (snake_case), par exemple `calculer_moyenne` ;
- noms de constantes et de macros en majuscules, par exemple `TAILLE_MAX` ;
- noms explicites plutôt que des abréviations cryptiques (`compteur_erreurs` plutôt que `ce`).

Les commentaires

```
// Commentaire sur une seule ligne

/* Commentaire
   sur plusieurs lignes */
```

Un commentaire ne doit pas répéter ce que le code dit déjà, mais expliquer *pourquoi* un choix a été fait, ou documenter une hypothèse importante (par exemple une précondition non vérifiée par le code lui-même). En sécurité, un commentaire signalant explicitement « cette fonction suppose que `buffer` fait au moins `TAILLE_MAX` octets » a une réelle valeur pour un futur relecteur ou auditeur du code.

À retenir

- Un algorithme se conçoit avant de se coder, et se raisonne en termes d'entrées, de sortie, de préconditions et de postconditions.

- Le C est un langage compilé, statiquement typé, proche de la machine, normalisé par l'ISO (C89 à C23).
- Le C ne protège pas automatiquement le programmeur : pas de vérification de bornes, pas de ramasse-miettes ; la rigueur doit donc venir du développeur et de ses outils.
- La chaîne de compilation comprend préprocesseur, compilation et édition de liens ; comprendre ces étapes aide à diagnostiquer les erreurs.
- Toujours compiler avec les avertissements activés (`-Wall` `-Wextra`, voire `-Werror` et les sanitizers) : la majorité des bugs de sécurité en C sont détectables dès la compilation ou par analyse statique/dynamique.

Chapitre 2 — Variables, types, opérateurs et entrées-sorties

1. Variables et types de base

Une variable est un emplacement mémoire nommé, dont le **type** détermine trois choses : la taille occupée en mémoire (en octets), l'interprétation binaire de son contenu (entier signé, flottant, caractère...), et l'ensemble des opérations autorisées sur elle. Déclarer une variable, c'est donc réserver un espace mémoire et indiquer au compilateur comment ce contenu doit être manipulé.

Type	Taille usuelle	Exemple	Rôle
<code>int</code>	4 octets	<code>int age = 25;</code>	entier signé
<code>unsigned int</code>	4 octets	<code>unsigned int compteur = 0;</code>	entier non signé
<code>char</code>	1 octet	<code>char lettre = 'A';</code>	caractère / petit entier
<code>float</code>	4 octets	<code>float prix = 19.99f;</code>	flottant simple précision
<code>double</code>	8 octets	<code>double pi = 3.14159265;</code>	flottant double précision
<code>long</code> , <code>short</code>	variable	<code>long total;</code>	variantes de taille de <code>int</code>
<code>_Bool</code> (<code>bool</code> via <code>stdbool.h</code>)	1 octet	<code>bool actif = true;</code>	valeur booléenne (0 ou 1)

La taille exacte de chaque type dépend de l'architecture et du compilateur : le standard C ne garantit que des tailles *minimales* relatives (par exemple `sizeof(short) <= sizeof(int) <= sizeof(long)`), pas des tailles absolues. On utilise l'opérateur `sizeof(type)` pour la vérifier concrètement :

```
#include <stdio.h>

int main(void) {
    printf("sizeof(int)      = %zu\n", sizeof(int));
    printf("sizeof(long)     = %zu\n", sizeof(long));
    printf("sizeof(double) = %zu\n", sizeof(double));
    return 0;
}
```

Pour des programmes dont le comportement doit être **prévisible et portable** — ce qui est un impératif en sécurité, où une taille de type ambiguë peut ouvrir une faille sur une architecture donnée et pas sur une autre — on préfère les types à taille garantie de `<stdint.h>` : `int8_t`, `uint8_t`, `int16_t`, `uint32_t`, `int64_t`, etc. Le suffixe indique le nombre de bits, et le préfixe `u` signale un type non signé.

```
#include <stdint.h>

uint8_t  octet    = 255;        // 0 à 255
int32_t  identifiant = -42;     // -2 147 483 648 à 2 147 483 647
uint32_t taille   = 4096;       // toujours 32 bits, quelle que soit l'architecture
```

Limites des types numériques

Les en-têtes `<limits.h>` (entiers) et `<float.h>` (flottants) définissent les bornes de chaque type, par exemple `INT_MAX`, `INT_MIN`, `UINT_MAX`. Connaître ces bornes est indispensable pour raisonner sur les **débordements d'entier** (*integer overflow*), abordés en détail plus bas.

2. Déclaration, initialisation, portée

```
int x;                // déclarée, valeur indéterminée (danger !)
int y = 0;            // déclarée et initialisée
const int MAX = 100;  // constante, non modifiable après initialisation
```

Une variable non initialisée contient une valeur indéterminée (souvent appelée « garbage » : ce qui se trouvait précédemment à cet emplacement mémoire). L'utiliser avant affectation est un comportement indéfini en C : le programme peut produire un résultat incohérent, planter, ou — plus insidieux — fonctionner « par chance » sur la machine de développement puis échouer en production. C'est également une source réelle de vulnérabilité : lire une zone mémoire non initialisée peut exposer des données résiduelles d'un traitement précédent (mot de passe, clé, fragment de fichier), une classe de vulnérabilité connue sous le nom de fuite d'information par mémoire non initialisée.

Le mot-clé `const` ne rend pas une variable « constante » au sens mathématique strict (elle occupe toujours un emplacement mémoire), mais interdit au compilateur d'accepter toute affectation ultérieure — une protection précieuse contre les modifications accidentelles.

```
const int LIMITE = 10;
LIMITE = 20; // erreur de compilation : assignment of read-only variable
```

Portée en bref

La portée d'une variable (l'endroit du programme où son nom est visible) est traitée en détail au chapitre 4 ; retenons pour l'instant qu'une variable déclarée à l'intérieur d'un bloc `{ ... }` (par exemple le corps de `main`, ou celui d'une boucle) n'existe que dans ce bloc.

3. Opérateurs

- **Arithmétiques** : `+` `-` `*` `/` `%` (le `%`, modulo, n'existe que pour les types entiers).
- **Relationnels** : `==` `!=` `<` `>` `<=` `>=` (le résultat est un entier valant 0 — faux — ou 1 — vrai).

- **Logiques** : `&&` `||` `!` (évaluation en court-circuit : dans `a && b`, si `a` est faux, `b` n'est pas évalué).
- **Bit à bit** : `&` `|` `^` `~` `<<` `>>` — fondamentaux en cryptographie (XOR, masques, décalages) et en manipulation bas niveau de protocoles réseau ; ils seront réutilisés en profondeur au chapitre Cryptographie.
- **Affectation composée** : `+=` `-=` `*=` `/=` `%=` `&=` `|=` `^=` `<<=` `>>=`.
- **Incrémentation/décrémentation** : `++` `--`, en forme préfixe (`++i`, incrémente puis évalue) ou postfixe (`i++`, évalue puis incrémente).

```
int a = 5, b = 3;
int somme = a + b;      // 8
int reste = a % b;     // 2
int masque = a & b;    // ET bit à bit -> 1
int i = 0;
int avant = i++;       // avant = 0, i vaut 1 ensuite
int apres = ++i;       // i vaut 2, apres = 2
```

Évaluation en court-circuit : un outil de sécurité défensive

L'évaluation en court-circuit de `&&` et `||` est souvent exploitée volontairement pour écrire du code défensif, en s'assurant qu'une condition dangereuse n'est testée que si une condition de garde préalable est vraie :

```
char *chaine = NULL;
if (chaine != NULL && chaine[0] == 'A') {
    // chaine[0] n'est évalué que si chaine != NULL est vrai :
    // aucun déréférencement de pointeur nul possible ici.
}
```

Inverser l'ordre des deux conditions (`chaine[0] == 'A' && chaine != NULL`) provoquerait un déréférencement de pointeur nul si `chaine` vaut `NULL` — un piège classique pour un débutant.

Débordement d'entier (*integer overflow*)

Un type entier ne peut représenter qu'un intervalle fini de valeurs. Dépasser cet intervalle produit un débordement :

```
#include <limits.h>
#include <stdio.h>

int main(void) {
    int x = INT_MAX;
    printf("%d\n", x + 1); // comportement indéfini pour un int signé !
    return 0;
}
```

Pour les entiers **signés**, un débordement est un comportement indéfini par la norme C (le compilateur est libre d'optimiser en supposant qu'il ne se produit jamais). Pour les entiers **non signés**, le

débordement est défini et « boucle » selon une arithmétique modulo 2^n (`UINT_MAX + 1` vaut 0). Cette différence de comportement est une source fréquente de vulnérabilités : un calcul de taille de tampon effectué en entier non signé peut « boucler » vers une petite valeur après une soustraction inattendue, provoquant ensuite une allocation trop petite et un débordement d'écriture.

4. Conversions de type (casting)

```
int i = 7;
double d = (double) i / 2;    // conversion explicite : 3.5
int tronque = (int) 3.9;      // conversion explicite : 3 (troncature, pas arrondi)
```

On distingue :

- la **conversion explicite** (*cast*), écrite `(type) expression`, décidée volontairement par le programmeur ;
- la **conversion implicite**, effectuée silencieusement par le compilateur, par exemple lors du passage d'un `int` à un `double` dans une expression mixte, ou d'un `int` à un `char` lors d'une affectation.

```
char c = 300;    // conversion implicite : dépasse la plage d'un char, résultat dépendant
int i = 3.99;    // conversion implicite : i vaut 3 (troncature)
```

Les conversions implicites peuvent provoquer des pertes de précision ou des dépassements silencieux, sans le moindre avertissement à l'exécution : c'est une source fréquente de bugs de sécurité, par exemple la troncature d'une taille de buffer calculée en `size_t` (non signé, souvent 64 bits) vers un `int` (signé, 32 bits), qui peut transformer une taille légitime en valeur négative ou aberrante une fois réinterprétée.

Bonne pratique : rendre les conversions explicites et vérifiées

```
#include <stdio.h>

int main(void) {
    long valeur = 5000000000L; // dépasse la plage d'un int sur la plupart des systèmes
    if (valeur > 2147483647L || valeur < -2147483648L) {
        fprintf(stderr, "Valeur hors plage pour un int\n");
    } else {
        int v = (int) valeur;
        printf("%d\n", v);
    }
    return 0;
}
```

Documenter et vérifier explicitement une conversion potentiellement destructrice est une habitude directement transposable à l'audit de code sécurisé : un relecteur doit pouvoir repérer d'un coup d'œil les points où une donnée change de représentation.

5. Entrées-sorties standard

```
#include <stdio.h>

int main(void) {
    int age;
    printf("Entrez votre âge : ");
    scanf("%d", &age);          // & : adresse de la variable
    printf("Vous avez %d ans.\n", age);
    return 0;
}
```

Spécificateurs de format courants : `%d` (int), `%u` (unsigned int), `%f` (float/double en écriture avec `printf`, `%lf` en lecture avec `scanf`), `%c` (char), `%s` (chaîne), `%p` (pointeur), `%x/%X` (hexadécimal), `%zu` (`size_t`, le type renvoyé par `sizeof`).

Vérifier la valeur de retour de `scanf`

`scanf` renvoie le nombre de conversions réussies. Une saisie inattendue (l'utilisateur tape des lettres alors qu'un entier est attendu) fait échouer la conversion sans que la variable de destination soit modifiée : ignorer la valeur de retour, c'est risquer d'utiliser ensuite une variable non initialisée.

```
int age;
if (scanf("%d", &age) != 1) {
    fprintf(stderr, "Saisie invalide\n");
    return 1;
}
```

Point de vigilance sécurité : chaînes de format et tampons

`scanf("%s", ...)` sans limite de taille, ou une chaîne de format contrôlée par l'utilisateur passée directement à `printf` (par exemple `printf(entree_utilisateur);` au lieu de `printf("%s", entree_utilisateur);`), sont des vulnérabilités classiques : respectivement un dépassement de tampon (l'utilisateur peut saisir un texte plus long que le buffer alloué) et une vulnérabilité de type *format string*, qui peut permettre de lire, voire d'écrire, des zones arbitraires de la mémoire du processus. La règle à retenir dès maintenant : **une chaîne de format doit toujours être un littéral écrit par le développeur, jamais une donnée fournie par l'utilisateur**. Ce point sera repris et exploité en détail dans le cours *Sécurité des applications*.

```
char nom[20];
printf("%s\n", nom);          // correct : format fixe, donnée en argument
printf(nom);                  // dangereux : nom devient la chaîne de format elle-même
```

À retenir

- Toujours initialiser ses variables ; une valeur « indéterminée » n'est jamais un état sûr.

- Connaître la taille et les limites de chaque type (`sizeof`, `limits.h`, `stdint.h`), en particulier la distinction entre débordement signé (indéfini) et non signé (défini, modulo).
- Distinguer conversions explicites (voulues, documentées) et implicites (silencieuses, à surveiller), notamment entre types signés et non signés.
- Vérifier systématiquement la valeur de retour de `scanf`.
- Se méfier des chaînes de format non maîtrisées : jamais de donnée utilisateur directement comme premier argument de `printf/scanf`.

Chapitre 3 — Structures de contrôle

1. Structures conditionnelles

Les structures conditionnelles permettent de faire dépendre l'exécution d'une ou plusieurs instructions de la valeur d'une expression booléenne. En C, toute expression peut être utilisée comme condition : elle est considérée fausse si elle vaut 0, vraie dans tous les autres cas. C'est une source fréquente d'erreurs subtiles (par exemple confondre `=` et `==`), détaillée plus bas.

`if / else if / else`

```
int note = 12;
if (note >= 16) {
    printf("Très bien\n");
} else if (note >= 10) {
    printf("Admis\n");
} else {
    printf("Ajourné\n");
}
```

Les blocs `{ }` sont facultatifs en C lorsque le corps du `if` (ou du `else`) ne comporte qu'une seule instruction, mais il est fortement recommandé de toujours les écrire, y compris pour une instruction unique. Cette discipline évite un piège classique :

```
if (utilisateur_authentifie)
    printf("Accès autorisé\n");
    ouvrir_session(); // INDENTÉ MAIS PAS DANS LE if ! s'exécute toujours
```

Dans cet exemple, l'indentation suggère que `ouvrir_session()` ne s'exécute que si l'utilisateur est authentifié, alors qu'en réalité elle s'exécute **systématiquement**, car seule l'instruction `printf` appartient au `if` en l'absence d'accolades. Ce type d'erreur, purement visuel, a été à l'origine de vulnérabilités réelles dans du code historique (le bug dit « goto fail » d'une célèbre implémentation TLS en est l'exemple le plus souvent cité, bien qu'il s'agisse d'un `goto` sans accolade plutôt que d'un `if`). La règle à retenir : **toujours utiliser des accolades**, même pour un bloc d'une seule ligne.

Piège classique : `=` au lieu de `==`

```
int mot_de_passe_ok = 0;
if (mot_de_passe_ok = 1) { // BUG : affectation, pas comparaison !
    printf("Accès autorisé\n"); // s'exécute toujours, quelle que soit la valeur initia
}
```

`mot_de_passe_ok = 1` est une expression d'affectation, qui vaut elle-même 1 (donc « vraie ») après exécution : la condition est donc toujours satisfaite, indépendamment de la logique métier attendue. La

plupart des compilateurs modernes émettent un avertissement pour ce genre de code avec `-Wall` (`-Wparentheses` notamment) : c'est une raison supplémentaire de toujours activer les avertissements de compilation.

switch

```
int jour = 3;
switch (jour) {
    case 1: printf("Lundi\n"); break;
    case 2: printf("Mardi\n"); break;
    case 3: printf("Mercredi\n"); break;
    default: printf("Jour invalide\n");
}
```

Le `switch` compare une expression entière (ou de type énumération) à une série de valeurs constantes. L'oubli d'un `break` provoque un **fall-through** : l'exécution continue dans le `case` suivant, sans re-tester sa condition. C'est parfois voulu (par exemple pour regrouper plusieurs cas qui partagent le même traitement), mais c'est le plus souvent une source de bug. Toujours documenter explicitement un fall-through intentionnel par un commentaire, par exemple `// fall-through volontaire`, afin qu'un futur relecteur ne le corrige pas par erreur.

```
switch (niveau_journalisation) {
    case NIVEAU_CRITIQUE:
    case NIVEAU_ERREUR:
        // fall-through volontaire : critique et erreur partagent le même traitement
        enregistrer_dans_le_journal_urgent();
        break;
    case NIVEAU_INFO:
        enregistrer_dans_le_journal_standard();
        break;
    default:
        break;
}
```

Le cas `default` n'est pas obligatoire syntaxiquement, mais l'omettre revient à supposer implicitement que toutes les valeurs possibles ont été énumérées : une hypothèse souvent fausse (nouvelle valeur ajoutée plus tard, valeur issue d'une entrée non validée). L'inclure systématiquement, même pour simplement journaliser un cas inattendu, est une bonne pratique défensive.

Opérateur ternaire

```
int max = (a > b) ? a : b;
```

L'opérateur ternaire `condition ? valeur_si_vrai : valeur_si_faux` est une expression, pas une instruction : il produit une valeur et peut donc être utilisé partout où une expression est attendue (initialisation, argument de fonction). Il est adapté aux choix simples entre deux valeurs ; au-delà, un `if/else` classique reste plus lisible.

2. Boucles

Une boucle répète un bloc d'instructions tant qu'une condition reste vraie. Le choix de la structure de boucle dépend essentiellement de la connaissance ou non, *a priori*, du nombre d'itérations.

while

```
int i = 0;
while (i < 5) {
    printf("%d\n", i);
    i++;
}
```

La condition est testée **avant** chaque itération : si elle est fausse dès le départ, le corps de la boucle ne s'exécute jamais.

do ... while

Exécute le corps au moins une fois avant de tester la condition — utile pour valider une saisie utilisateur, car il faut bien demander la saisie au moins une fois avant de pouvoir la vérifier.

```
int choix;
do {
    printf("Choix (1-3) : ");
    scanf("%d", &choix);
} while (choix < 1 || choix > 3);
```

Cet exemple mérite d'être complété, en pratique, par une vérification de la valeur de retour de `scanf` (voir chapitre 2) : si l'utilisateur saisit une entrée non numérique, `scanf` échoue et `choix` peut conserver une valeur indéterminée à la première itération, ou provoquer une boucle infinie si le flux d'entrée n'est jamais « vidé » des caractères invalides.

for

```
for (int i = 0; i < 10; i++) {
    printf("%d ", i);
}
```

La boucle `for` regroupe en une seule ligne l'initialisation, la condition de poursuite et la mise à jour du compteur, ce qui la rend particulièrement adaptée lorsque le nombre d'itérations est connu ou calculable à l'avance (parcours de tableau, répétition d'un nombre fixe d'opérations).

```
for (initialisation ; condition ; mise_à_jour) {
    corps de la boucle
}
```

Les trois expressions sont facultatives (`for (;;)` est une boucle infinie valide), et la portée d'une variable déclarée dans l'initialisation (`int i`) est limitée au corps de la boucle depuis le standard C99.

3. Rupture de séquence

- `break` : sort immédiatement de la boucle englobante (ou du `switch`), sans exécuter le reste du bloc.
- `continue` : passe directement à l'itération suivante, en sautant le reste du corps de la boucle courante (mais en réévaluant la condition ou, pour un `for`, en exécutant d'abord la mise à jour).
- `goto` : transfère l'exécution vers une étiquette (`label:`) du même fichier. À éviter en général, car il nuit gravement à la lisibilité et à la prévisibilité du flot de contrôle, sauf cas très spécifiques (sortie d'imbrications profondes de boucles, ou gestion d'erreurs façon « cleanup » en C bas niveau, un idiomme courant dans le code noyau Linux par exemple).

```
// Idiomme "goto cleanup", courant en C bas niveau
FILE *f = fopen("donnees.txt", "r");
if (f == NULL) goto erreur;

int *tampon = malloc(100 * sizeof(int));
if (tampon == NULL) goto fermer_fichier;

// ... traitement ...

free(tampon);
fermer_fichier:
    fclose(f);
    return 0;
erreur:
    return 1;
```

Cet idiomme, bien que dérogeant à la règle générale « éviter `goto` », est largement accepté car il centralise la libération des ressources en un seul endroit, évitant la duplication de code de nettoyage à chaque point de sortie en erreur — une préoccupation directement liée à la prévention des fuites de ressources.

4. Boucles infinies et conditions de terminaison

```
for (;;) {
    // boucle infinie volontaire, sortie via break
}
```

Une condition de terminaison mal posée (par exemple un compteur jamais incrémenté, une condition toujours vraie, ou une comparaison qui ne peut jamais devenir fausse à cause d'un débordement d'entier) provoque une boucle qui ne se termine jamais, entraînant un déni de service par consommation de ressources (temps CPU, voire mémoire si la boucle alloue). Ce cas de figure — une entrée utilisateur qui provoque une boucle infinie ou une consommation excessive de ressources — est un cas particulier à garder en tête pour le cours *Security by design*, où l'on distingue les vulnérabilités

qui compromettent la confidentialité ou l'intégrité de celles qui compromettent uniquement la disponibilité.

Piège : boucle avec compteur non signé décroissant

```
for (unsigned int i = 10; i >= 0; i--) {  
    printf("%u\n", i);  
} // boucle infinie : i >= 0 est toujours vrai pour un unsigned int (i "boucle" à un tr
```

Comme vu au chapitre 2, un type non signé ne peut jamais être négatif : la condition `i >= 0` est donc toujours vraie, et lorsque `i` atteint 0 puis est décrémenté, elle « boucle » vers la plus grande valeur représentable au lieu de devenir négative. Ce genre d'erreur, purement liée au choix du type, illustre pourquoi il faut choisir ses types entiers avec autant de soin que ses conditions de boucle.

5. Complexité algorithmique (introduction)

Le nombre d'itérations d'une boucle détermine la complexité temporelle d'un algorithme, notée en notation de Landau (grand O), qui exprime comment le temps d'exécution croît en fonction de la taille n des données, indépendamment des détails d'implémentation ou de la machine utilisée. Exemples :

- une recherche séquentielle dans un tableau de taille n est en $O(n)$: dans le pire cas, il faut examiner chaque élément une fois ;
- une recherche dichotomique dans un tableau **trié** est en $O(\log n)$: chaque comparaison élimine la moitié des candidats restants ;
- une boucle imbriquée dans une autre (par exemple pour comparer chaque paire d'éléments d'un tableau) est typiquement en $O(n^2)$.

```
// Complexité O(n) : une seule boucle, n itérations  
for (int i = 0; i < n; i++) { /* ... */ }  
  
// Complexité O(n * n) = O(n²) : boucle imbriquée  
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++) { /* ... */ }  
}
```

Cette notion, encore intuitive à ce stade, sera systématiquement réutilisée pour analyser le coût des algorithmes cryptographiques (chapitre Cryptographie) et pour évaluer la faisabilité pratique des attaques par force brute (chapitre Cryptanalyse) : un espace de clés dont l'exploration exhaustive est en $O(2^n)$ devient rapidement impraticable lorsque n augmente, ce qui est précisément le principe de sécurité recherché par les algorithmes de chiffrement modernes.

À retenir

- Choisir la structure adaptée : `for` pour un nombre d'itérations connu, `while`/`do while` pour une condition dynamique évaluée avant/après le corps.

- Toujours utiliser des accolades, même pour un bloc d'une seule instruction, afin d'éviter les erreurs d'indentation trompeuse.
- Ne jamais confondre `=` (affectation) et `==` (comparaison) dans une condition ; s'appuyer sur les avertissements du compilateur pour les détecter.
- `switch` sans `break` = piège classique de fall-through ; toujours prévoir un `default`.
- Toujours s'assurer qu'une boucle progresse réellement vers sa condition d'arrêt, en particulier avec des compteurs non signés.

Chapitre 4 — Fonctions et portée des variables

1. Pourquoi découper en fonctions ?

Une fonction encapsule un traitement réutilisable, sous un nom qui décrit son intention. Ce découpage apporte plusieurs bénéfices concrets :

- **Lisibilité** : un programme découpé en fonctions bien nommées se lit presque comme une table des matières (`lire_configuration()`, `valider_entree()`, `calculer_resultat()`).
- **Réutilisabilité** : une fonction correctement écrite peut être appelée depuis plusieurs endroits du programme, sans dupliquer le code.
- **Testabilité** : une fonction avec des entrées et une sortie clairement définies peut être testée isolément, indépendamment du reste du programme.
- **Limitation de la portée des erreurs** : un bug dans une fonction bien délimitée est plus facile à localiser qu'un bug perdu dans une longue suite d'instructions au sein d'un unique `main`.

C'est un principe de base du *Security by design* : plus une fonction est petite et ciblée (elle ne fait « qu'une seule chose »), plus elle est facile à auditer, à tester exhaustivement, et à faire évoluer sans introduire de régression. Un code composé de fonctions de plusieurs centaines de lignes, aux responsabilités mélangées, est statistiquement bien plus difficile à sécuriser qu'un code découpé en unités courtes et cohérentes.

2. Déclaration, prototype, définition

```
// Prototype (souvent dans un .h)
int addition(int a, int b);

int main(void) {
    int resultat = addition(3, 4);
    printf("%d\n", resultat);
    return 0;
}

// Définition
int addition(int a, int b) {
    return a + b;
}
```

Le **prototype** (ou déclaration) annonce au compilateur la signature d'une fonction — son nom, le type de sa valeur de retour, et le nombre et le type de ses paramètres — sans fournir son corps. Il permet d'appeler une fonction avant sa définition complète dans le fichier, et surtout de vérifier la cohérence des types passés lors de chaque appel : le compilateur détecte alors des erreurs (mauvais nombre d'arguments, type incompatible) qu'une absence de prototype laisserait passer silencieusement, avec un comportement indéfini à l'exécution.

Dans un programme composé de plusieurs fichiers source, les prototypes des fonctions destinées à être partagées sont regroupés dans un fichier d'en-tête (.h), inclus par chaque fichier .c qui en a besoin :

```
// operations.h
#ifndef OPERATIONS_H
#define OPERATIONS_H

int addition(int a, int b);
int soustraction(int a, int b);

#endif
```

Les directives `#ifndef` / `#define` / `#endif` forment un **garde d'inclusion** (*include guard*) : elles empêchent le contenu du fichier d'en-tête d'être inséré plusieurs fois si plusieurs fichiers .c l'incluent, ce qui provoquerait des erreurs de redéfinition.

3. Passage de paramètres

- **Passage par valeur** (comportement par défaut en C, pour tous les types de base) : la fonction reçoit une **copie** de la valeur de l'argument ; toute modification à l'intérieur de la fonction est locale et n'affecte pas la variable de l'appelant.
- **Passage par adresse** (en transmettant un pointeur vers la variable) : permet à la fonction de modifier la variable de l'appelant, puisqu'elle reçoit non pas une copie de la valeur mais l'adresse mémoire où cette valeur est stockée.

```
void incrementer_par_valeur(int x) {
    x++;
    // ne modifie que la copie locale
}

void incrementer_par_adresse(int *x) {
    (*x)++;
    // modifie la variable pointée
}

int main(void) {
    int n = 5;
    incrementer_par_valeur(n);
    printf("%d\n", n);    // affiche toujours 5

    incrementer_par_adresse(&n);
    printf("%d\n", n);    // affiche 6
    return 0;
}
```

Cette distinction est fondamentale et sera approfondie au chapitre 6 (pointeurs), mais elle a déjà des implications en matière de conception sécurisée : une fonction qui prend un pointeur en paramètre peut, si elle est mal écrite, modifier ou lire au-delà de la zone mémoire prévue par l'appelant. Documenter clairement, pour chaque paramètre pointeur, ce que la fonction attend en entrée (taille du

tampon pointé, état attendu) et ce qu'elle garantit en sortie fait partie des bonnes pratiques attendues dans ce cours.

Paramètres constants : documenter l'intention de ne pas modifier

```
void afficher(const char *message) {  
    printf("%s\n", message);  
    // message[0] = 'X'; // interdit par le compilateur : message est const  
}
```

Préfixer un paramètre pointeur de `const` lorsque la fonction ne doit pas modifier les données pointées est une convention précieuse : elle documente l'intention dans la signature elle-même, et le compilateur la fait respecter — un exemple concret de « faire porter la contrainte de sécurité par le langage plutôt que par la seule discipline du développeur ».

4. Portée (scope) et durée de vie

Type de variable	Portée	Durée de vie
locale	bloc où elle est déclarée	jusqu'à la fin du bloc (pile)
globale	tout le fichier (ou tout le programme si non static)	toute l'exécution
static locale	bloc de déclaration	toute l'exécution, valeur conservée entre appels
paramètre	corps de la fonction	durée de l'appel

La **portée** (*scope*) définit où, dans le texte du programme, le nom d'une variable est visible et utilisable. La **durée de vie** définit pendant combien de temps, à l'exécution, l'emplacement mémoire associé à la variable reste valide. Ces deux notions ne coïncident pas toujours, ce qui est justement la source de certaines erreurs classiques (voir le pointeur pendant, abordé au chapitre 6).

```
int compteur_appels(void) {  
    static int n = 0; // initialisée une seule fois, à la première exécution  
    n++;  
    return n;  
}  
  
int main(void) {  
    printf("%d\n", compteur_appels()); // 1  
    printf("%d\n", compteur_appels()); // 2  
    printf("%d\n", compteur_appels()); // 3  
    return 0;  
}
```

Une variable `static` locale n'est initialisée qu'une seule fois (avant le premier appel de la fonction) et conserve sa valeur d'un appel à l'autre — contrairement à une variable locale ordinaire, réinitialisée à chaque appel et détruite à la fin de chaque exécution du bloc.

Variables globales : commodité contre risque

```
int solde_global = 0; // variable globale : visible et modifiable depuis tout le fichier

void deposter(int montant) {
    solde_global += montant; // effet de bord sur une variable globale
}
```

Les variables globales facilitent le partage d'état entre plusieurs fonctions sans avoir à le faire transiter explicitement par des paramètres. Mais elles nuisent fortement à la prévisibilité du code : n'importe quelle fonction du fichier (voire du programme, si la variable n'est pas `static`) peut la modifier, ce qui complique considérablement l'analyse de sécurité — on parle d'**effets de bord non locaux**, difficiles à tracer lors d'un audit ou d'un débogage. La bonne pratique consiste à limiter au strict nécessaire l'usage de variables globales, et à restreindre leur portée au fichier courant avec le mot-clé `static` lorsqu'elles n'ont pas besoin d'être visibles ailleurs :

```
static int compteur_interne = 0; // static au niveau fichier : invisible depuis les autres fichiers
```

Attention : le mot-clé `static` a ici un sens différent de celui vu pour une variable locale — au niveau global, il restreint la **portée** (liaison interne au fichier) plutôt que de modifier la **durée de vie** (qui est déjà celle du programme entier pour une variable globale).

5. Récursivité

```
unsigned long factorielle(unsigned int n) {
    if (n == 0) return 1; // cas de base : arrête la récursion
    return n * factorielle(n - 1); // appel récursif
}
```

Une fonction récursive s'appelle elle-même, directement ou indirectement, jusqu'à atteindre un **cas de base** qui arrête la récursion. Toute fonction récursive doit garantir deux propriétés : l'existence d'un cas de base atteignable, et la progression vers ce cas de base à chaque appel (ici, `n` décroît strictement).

Chaque appel récursif empile un nouveau cadre d'appel (*stack frame*) sur la pile d'exécution, contenant notamment les paramètres et variables locales de cet appel. Une récursion non bornée (cas de base jamais atteint, à cause d'une erreur de logique) ou simplement trop profonde (des milliers d'appels imbriqués sur une entrée légitime mais très grande) provoque un **débordement de pile** (*stack overflow*), qui termine brutalement le programme. C'est un cas concret de vulnérabilité par épuisement de ressource : une fonction récursive qui traite une structure de données fournie par l'utilisateur (par exemple un document imbriqué) sans limiter la profondeur de récursion peut être utilisée pour provoquer un déni de service.

```
// Version itérative équivalente, sans consommation de pile supplémentaire
unsigned long factorielle_iterative(unsigned int n) {
    unsigned long resultat = 1;
    for (unsigned int i = 2; i <= n; i++) {
        resultat *= i;
    }
    return resultat;
}
```

Lorsque la profondeur de récursion peut être contrôlée par une entrée externe, il est prudent soit de borner explicitement cette profondeur, soit de préférer une version itérative, plus sûre vis-à-vis de la consommation de pile.

6. Bonnes pratiques

- Une fonction = une responsabilité claire, exprimée par son nom.
- Toujours valider les paramètres reçus (pointeurs non nuls, tailles cohérentes, valeurs dans les bornes attendues) avant de les utiliser — ne jamais supposer qu'un appelant respecte automatiquement le contrat implicite d'une fonction.
- Documenter précondition et postcondition, y compris qui est responsable de libérer une mémoire éventuellement allouée par la fonction (l'appelant ou la fonction elle-même).
- Limiter le nombre de paramètres d'une fonction ; au-delà de quatre ou cinq, envisager de les regrouper dans une structure (voir chapitre 7).
- Préférer plusieurs petites fonctions bien nommées à une seule fonction longue mêlant plusieurs responsabilités.

À retenir

- Le passage par adresse (via pointeur) est le mécanisme C permettant à une fonction de modifier les données de l'appelant ; le passage par valeur, par défaut, ne le permet jamais pour les types de base.
- La portée `static` locale crée un état persistant discret entre les appels : à utiliser avec parcimonie et à documenter clairement.
- Les variables globales facilitent le partage d'état mais introduisent des effets de bord non locaux, difficiles à auditer.
- La récursivité est élégante et naturelle pour certains problèmes, mais bornée par la taille de la pile : toujours s'assurer qu'un cas de base est atteignable et que la profondeur reste raisonnable.

Chapitre 5 — Tableaux et chaînes de caractères

1. Tableaux statiques

```
int notes[5] = {12, 15, 8, 17, 10};
int i;
for (i = 0; i < 5; i++) {
    printf("%d\n", notes[i]);
}
```

En C, un tableau `notes[5]` occupe un bloc **contigu** de mémoire : les cinq entiers sont stockés côte à côte, sans espace ni structure supplémentaire. C'est cette contiguïté qui rend l'accès indexé aussi rapide : `notes[i]` est strictement équivalent à `*(notes + i)`, c'est-à-dire « ajouter `i` fois la taille d'un `int` à l'adresse de début du tableau, puis lire la valeur à cette adresse ». Cette équivalence sera reprise et approfondie au chapitre 6 sur les pointeurs.

Le C n'effectue aucune vérification de borne, ni à la compilation, ni à l'exécution. Lire ou écrire `notes[5]` ou `notes[-1]` compile sans erreur et s'exécute, en accédant à de la mémoire qui se trouve juste après ou juste avant le tableau — mémoire qui appartient potentiellement à une autre variable, ou qui n'est même pas allouée au processus. C'est la cause première des débordements de tampon (*buffer overflow*), l'une des vulnérabilités logicielles les plus exploitées historiquement dans les systèmes écrits en C : elle sera approfondie, avec des exemples d'exploitation, dans le cours *Sécurité des applications*.

```
int notes[5] = {12, 15, 8, 17, 10};
notes[5] = 99; // compile ! écrit hors du tableau : comportement indéfini
printf("%d\n", notes[10]); // compile ! lit hors du tableau : valeur imprévisible
```

Initialisation et taille implicite

```
int a[5] = {1, 2, 3}; // les 2 derniers éléments sont initialisés à 0
int b[] = {1, 2, 3, 4}; // taille déduite automatiquement : 4
int c[5] = {0}; // tous les éléments à 0
```

Lorsqu'une initialisation partielle est fournie, les éléments non explicitement initialisés sont mis à zéro — mais uniquement dans le cas d'une initialisation *avec accolades*. Un tableau local déclaré sans initialisation du tout (`int a[5];`) contient, comme pour une simple variable, des valeurs indéterminées.

sizeof et taille d'un tableau

```
int notes[5];
printf("%zu\n", sizeof(notes)); // taille totale en octets (5 * sizeof(int))
printf("%zu\n", sizeof(notes) / sizeof(notes[0])); // nombre d'éléments : 5
```

Cette technique (`sizeof(tableau) / sizeof(tableau[0])`) ne fonctionne que tant que `notes` désigne réellement le tableau complet, dans la portée où il a été déclaré. Dès qu'un tableau est transmis à une fonction (voir plus bas), cette astuce cesse de fonctionner : c'est l'un des pièges les plus fréquents pour un programmeur C débutant.

2. Tableaux à deux dimensions

```
int matrice[3][4];
matrice[1][2] = 7;
```

Un tableau à deux dimensions est stocké en mémoire en *row-major order* (ordre ligne par ligne) : les éléments d'une même ligne sont contigus, et la ligne suivante commence immédiatement après la fin de la précédente. Concrètement, `matrice[3][4]` occupe le même espace mémoire contigu qu'un tableau à une dimension de 12 éléments, et `matrice[i][j]` est équivalent à un accès à l'indice `i * 4 + j` dans ce tableau linéaire.

```
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 4; j++) {
        matrice[i][j] = i * 4 + j;
    }
}
```

3. Tableaux passés en paramètre de fonction

Lorsqu'un tableau est passé en argument à une fonction, il est en réalité converti (« dégradé ») en un simple pointeur vers son premier élément : la fonction ne reçoit **aucune information sur sa taille**.

```
void afficher_tableau(int t[], int taille) {
    // ici, sizeof(t) donnerait la taille d'un pointeur (souvent 8), pas celle du tableau
    for (int i = 0; i < taille; i++) {
        printf("%d ", t[i]);
    }
    printf("\n");
}

int main(void) {
    int notes[5] = {12, 15, 8, 17, 10};
    afficher_tableau(notes, 5); // la taille doit être transmise explicitement
    return 0;
}
```

Cette caractéristique du langage — un tableau C ne connaît pas sa propre taille à l'exécution, et cette information se « perd » dès qu'il est transmis à une fonction — oblige à transmettre systématiquement la taille en paramètre séparé. Oublier de le faire, ou transmettre une taille incorrecte, est une cause directe de débordement de tampon dans des fonctions qui manipulent des tableaux « à l'aveugle ».

4. Chaînes de caractères

En C, une chaîne est un tableau de `char` terminé par l'octet nul `'\0'` (valeur 0), qui marque la fin logique de la chaîne indépendamment de la taille physique du tableau qui la contient.

```
char nom[20] = "Zakaria";
printf("%s\n", nom);
printf("Longueur : %zu\n", strlen(nom)); // 7, sans compter le '\0' final
```

Le tableau `nom` occupe 20 octets, mais la chaîne qu'il contient ne fait que 7 caractères utiles + 1 octet nul ; les 12 octets restants sont inutilisés (ou contiennent des valeurs résiduelles, selon le mode d'initialisation). C'est précisément cette convention — la longueur d'une chaîne est déterminée par la position du `'\0'`, pas par la taille du tableau — qui rend les fonctions de manipulation de chaînes potentiellement dangereuses : rien n'empêche une fonction d'écrire au-delà des limites physiques du tableau en cherchant, ou en plaçant, cet octet nul.

Fonctions de `string.h`

Fonction	Rôle	Risque
<code>strlen</code>	longueur d'une chaîne	boucle indéfiniment (et lit hors mémoire) si le <code>'\0'</code> est absent
<code>strcpy(dst, src)</code>	copie	aucune vérification de taille → débordement si <code>src</code> est plus long que <code>dst</code>
<code>strncpy(dst, src, n)</code>	copie bornée à <code>n</code> octets	ne garantit pas la terminaison par <code>'\0'</code> si <code>src</code> fait <code>n</code> caractères ou plus
<code>strcat(dst, src)</code>	concaténation	même risque que <code>strcpy</code> : aucune vérification de la taille disponible dans <code>dst</code>
<code>strncat(dst, src, n)</code>	concaténation bornée	plus sûre, mais <code>n</code> doit être calculé avec soin (espace réellement disponible)
<code>strcmp(a, b)</code>	comparaison lexicographique	— (renvoie 0 si égales, une valeur non nulle sinon)
<code>snprintf</code>	formatage borné vers une chaîne	alternative sûre recommandée, y compris pour construire une chaîne à partir de plusieurs valeurs

```
char buffer[10];
strcpy(buffer, "Ceci dépasse dix caractères"); // comportement indéfini : écrase la mém
```

Dans cet exemple, la chaîne source fait bien plus de 10 caractères : `strcpy` continue d'écrire au-delà des limites de `buffer`, corrompant potentiellement d'autres variables locales, l'adresse de retour de la fonction sur la pile, ou toute autre donnée voisine en mémoire. C'est l'archétype du débordement de tampon exploitable.

strncpy : un piège fréquent malgré son nom rassurant

```
char destination[10];
strncpy(destination, "Un texte assez long", sizeof(destination));
// destination n'est PAS terminée par '\0' ici : les 10 octets sont remplis
// sans qu'aucun octet nul n'ait pu être copié.
printf("%s\n", destination); // comportement indéfini : strlen/printf liront au-delà du
```

`strncpy` copie au plus `n` octets, mais si la source est plus longue (ou de longueur égale) que `n`, elle **ne termine pas** la destination par `'\0'`. Toute fonction utilisée ensuite sur `destination` en supposant qu'il s'agit d'une chaîne C valide (comme `printf("%s", ...)` ou `strlen`) lira alors au-delà du tableau. La bonne pratique consiste à forcer explicitement la terminaison :

```
char destination[10];
strncpy(destination, "Un texte assez long", sizeof(destination) - 1);
destination[sizeof(destination) - 1] = '\0'; // terminaison garantie
```

Bonne pratique : privilégier `snprintf`

```
char message[50];
snprintf(message, sizeof(message), "Utilisateur : %s (id=%d)", nom, id);
```

`snprintf` prend explicitement la taille du buffer de destination et garantit qu'elle ne sera jamais dépassée, tout en garantissant la terminaison par `'\0'` (sauf cas de taille nulle). C'est l'une des fonctions les plus sûres et les plus flexibles pour construire des chaînes en C, et elle doit être préférée systématiquement à `strcpy/strcat` dans du code nouveau.

5. Tableaux de chaînes

```
const char *jours[] = {"Lundi", "Mardi", "Mercredi"};
printf("%s\n", jours[1]); // Mardi
```

Ici, `jours` est un tableau de pointeurs vers des chaînes littérales (stockées en mémoire statique, en lecture seule). Le mot-clé `const` est important : une chaîne littérale ne doit jamais être modifiée via un pointeur non constant, car cela constitue un comportement indéfini (certaines plateformes stockent effectivement les littéraux en mémoire protégée en écriture).

6. Parcours et algorithmes classiques

- **Recherche** d'un élément : recherche séquentielle ($O(n)$, fonctionne sur tout tableau), ou recherche dichotomique sur tableau **trié** ($O(\log n)$, en divisant à chaque étape l'intervalle de recherche par deux).
- **Tri** : tri par sélection, tri par insertion, tri à bulles (introduction pédagogique aux algorithmes de tri, tous en $O(n^2)$ dans le pire cas), complétés par des algorithmes plus efficaces — tri rapide

(*quicksort*), tri fusion (*mergesort*), tous deux en $O(n \log n)$ en moyenne — étudiés en travaux pratiques.

- **Inversion** d'un tableau en place, par la technique dite des deux pointeurs (un indice au début, un à la fin, qui se rapprochent en échangeant les éléments) : une technique élémentaire mais réutilisée dans des contextes très variés, y compris en cryptographie pour l'endianness (l'ordre des octets d'un nombre) et le padding (remplissage) de blocs de données.

```
void inverser(int t[], int taille) {
    int debut = 0, fin = taille - 1;
    while (debut < fin) {
        int temp = t[debut];
        t[debut] = t[fin];
        t[fin] = temp;
        debut++;
        fin--;
    }
}
```

À retenir

- Un tableau C ne connaît pas sa propre taille à l'exécution : il faut la transmettre séparément, notamment lors du passage en paramètre de fonction, où il « dégrade » en simple pointeur.
- Aucune vérification de borne, ni à la compilation ni à l'exécution : la responsabilité de la validité des indices incombe entièrement au programmeur.
- Une chaîne C est un tableau de `char` terminé par `'\0'` ; toute fonction qui manipule des chaînes sans vérifier les tailles disponibles (`strcpy`, `strcat`, `strncpy` mal utilisée) est un point d'attention sécurité de premier plan.
- Préférer systématiquement les fonctions bornées et sûres (`snprintf`, `strncat` avec vérification manuelle du `'\0'` final) et toujours vérifier la taille des buffers avant toute copie.

Chapitre 6 — Pointeurs et gestion mémoire

1. Qu'est-ce qu'un pointeur ?

Un pointeur est une variable dont la valeur est une adresse mémoire — c'est-à-dire l'emplacement où une autre donnée est stockée, plutôt que la donnée elle-même. Un pointeur est toujours typé : un `int *` pointe vers un emplacement censé contenir un `int`, ce qui permet au compilateur de savoir combien d'octets lire ou écrire, et comment interpréter ces octets.

```
int x = 10;
int *p = &x;      // p contient l'adresse de x
printf("%d\n", *p); // déréférencement : affiche 10
*p = 20;          // modifie x via le pointeur : x vaut 20 après cette ligne
```

- `&x` : opérateur « adresse de », renvoie l'adresse mémoire où est stockée `x`.
- `*p` : opérateur de déréférencement (« valeur pointée par »), lit ou écrit la donnée à l'adresse contenue dans `p`.

Cette double nature de `*` (déclaration d'un pointeur vs déréférencement dans une expression) est une source fréquente de confusion pour les débutants : `int *p;` déclare `p` comme un pointeur vers un `int`, tandis que `*p = 20;` déréférence `p` pour écrire à l'adresse qu'il contient.

Le pointeur nul

```
int *p = NULL; // p ne pointe vers rien de valide
if (p != NULL) {
    printf("%d\n", *p);
} else {
    printf("p ne pointe vers rien\n");
}
```

`NULL` est une valeur conventionnelle signifiant « ce pointeur ne référence aucune adresse valide ». Déréférencer un pointeur nul (`*p` alors que `p == NULL`) est un comportement indéfini qui provoque, en pratique, l'arrêt brutal du programme sur la plupart des systèmes (violation d'accès mémoire). Vérifier systématiquement qu'un pointeur n'est pas nul avant de le déréférencer, en particulier lorsqu'il provient du retour d'une fonction susceptible d'échouer (`malloc`, `fopen`, une fonction de recherche qui peut ne rien trouver), est une habitude essentielle.

2. Les quatre zones de la mémoire d'un programme

Zone	Contenu	Durée de vie
Code (text)	instructions du programme, généralement en lecture seule	statique, toute l'exécution

Zone	Contenu	Durée de vie
Données statiques	variables globales et <code>static</code> , initialisées ou non	toute l'exécution
Tas (heap)	allocation dynamique (<code>malloc</code> , <code>calloc</code> , <code>realloc</code>)	jusqu'à <code>free</code> explicite
Pile (stack)	variables locales, paramètres, adresses de retour des appels de fonction	durée du bloc / de l'appel

Comprendre cette organisation est indispensable pour analyser un débordement de pile, un *use-after-free*, ou une fuite mémoire, car chaque classe de vulnérabilité correspond à une manipulation incorrecte d'une de ces zones :

- un débordement de tampon sur un tableau local déborde dans la **pile**, où peuvent se trouver d'autres variables locales et, plus loin, l'adresse de retour de la fonction ;
- un *use-after-free* ou une fuite mémoire concerne le **tas**, dont la gestion est entièrement manuelle en C ;
- une écriture dans une chaîne littérale (mémoire statique en lecture seule) provoque une violation d'accès.

La pile croît et décroît automatiquement au rythme des appels et retours de fonction (chapitre 4) ; le tas, en revanche, ne connaît aucune gestion automatique en C standard : chaque allocation doit être explicitement libérée par le programmeur.

3. Allocation dynamique

```
#include <stdlib.h>

int *tableau = malloc(10 * sizeof(int));
if (tableau == NULL) {
    // échec d'allocation : toujours vérifier, ne jamais supposer que malloc réussit
    exit(EXIT_FAILURE);
}
for (int i = 0; i < 10; i++) tableau[i] = i * i;

free(tableau);
tableau = NULL; // évite un « dangling pointer » (pointeur pendan)
```

- `malloc(taille)` : alloue `taille` octets sur le tas, dont le contenu initial n'est **pas** défini (contrairement à `calloc`) — l'utiliser sans initialiser explicitement les données revient au même risque qu'une variable locale non initialisée.
- `calloc(n, taille)` : alloue `n * taille` octets et les initialise à zéro ; calcule également en interne le produit `n * taille` en se protégeant d'un éventuel débordement d'entier, ce que ne fait pas un appel manuel `malloc(n * taille)`.
- `realloc(ptr, nouvelle_taille)` : redimensionne un bloc existant, en peut le déplacer ; renvoie un nouveau pointeur qu'il faut impérativement récupérer (voir piège ci-dessous).

- `free(ptr)` : libère la mémoire précédemment allouée ; **ne jamais l'appeler deux fois sur le même pointeur** (*double free*) ni utiliser le pointeur après (*use-after-free*).

Piège classique avec `realloc`

```
int *tableau = malloc(10 * sizeof(int));
// ...
int *tmp = realloc(tableau, 20 * sizeof(int));
if (tmp == NULL) {
    // échec : tableau reste valide et doit toujours être libéré
    free(tableau);
    exit(EXIT_FAILURE);
}
tableau = tmp; // ne réaffecter tableau qu'après avoir vérifié le succès
```

Écrire directement `tableau = realloc(tableau, ...)` est dangereux : si `realloc` échoue, elle renvoie `NULL` **sans libérer le bloc original**, et cette affectation directe écrase alors l'unique pointeur vers ce bloc, provoquant une fuite mémoire définitive (le bloc original devient inaccessible, donc impossible à libérer).

4. Classes de vulnérabilités mémoire liées aux pointeurs

Vulnérabilité	Cause
Déréférencement de pointeur nul	oubli de vérifier le retour de <code>malloc/fopen</code> , ou variable pointeur non initialisée
Fuite mémoire (<i>memory leak</i>)	<code>malloc</code> (ou équivalent) sans <code>free</code> correspondant, ou pointeur écrasé avant libération
Use-after-free	utilisation (lecture, écriture, ou nouveau <code>free</code>) d'un pointeur après que la mémoire pointée a été libérée
Double free	<code>free</code> appelé deux fois sur le même pointeur, corrompant les structures internes de l'allocateur
Dangling pointer (pointeur pendant)	pointeur conservé vers une variable locale dont la portée est terminée, ou vers une zone déjà libérée
Buffer overflow	écriture (ou lecture) au-delà de la mémoire réellement allouée, via arithmétique de pointeur ou indexation de tableau

```

int *pointeur_pendant(void) {
    int local = 42;
    return &local; // ATTENTION : adresse d'une variable locale qui va être détruite au
}

int main(void) {
    int *p = pointeur_pendant();
    printf("%d\n", *p); // comportement indéfini : local n'existe plus sur la pile
    return 0;
}

```

Dans cet exemple, `local` est une variable de la pile : son emplacement mémoire redevient disponible dès le retour de la fonction `pointeur_pendant`, alors que l'appelant continue de détenir son adresse. Le comportement observé peut sembler correct « par chance » (la valeur 42 peut encore s'y trouver, si rien ne l'a écrasée entre-temps), ce qui rend ce type de bug particulièrement traître : il peut passer inaperçu en test et se manifester de façon imprévisible en production, y compris de façon exploitable si un attaquant parvient à contrôler ce qui réutilise cette zone mémoire.

Ces classes de vulnérabilités constituent le socle du cours *Sécurité des applications* (exploitation) et sont directement liées aux outils d'audit statique/dynamique vus en *Audit organisation et technique* : des outils comme AddressSanitizer (`-fsanitize=address`, voir chapitre 1) ou Valgrind sont conçus précisément pour détecter ces erreurs à l'exécution, bien avant qu'elles ne soient exploitées.

5. Arithmétique des pointeurs

```

int tab[5] = {10, 20, 30, 40, 50};
int *p = tab; // p pointe sur tab[0] (un tableau "dégrade" en pointeur vers son pr
printf("%d\n", *(p + 2)); // 30, équivalent à tab[2]
p++; // p pointe maintenant sur tab[1]
printf("%d\n", *p); // 20

```

L'arithmétique de pointeur se fait en unités de la **taille du type pointé**, pas en octets bruts : `p + 1` avance l'adresse de `sizeof(*p)` octets (4 octets pour un `int` classique), pas d'un seul octet. C'est ce mécanisme qui rend l'équivalence `tab[i] ↔ *(tab + i)` cohérente quel que soit le type des éléments.

Différence entre pointeur et tableau

Bien que `tab` et `p` puissent être utilisés de façon très similaire (`tab[i]`, `p[i]`, `*(tab + i)`, `*(p + i)` sont toutes des écritures valides), `tab` et `p` ne sont pas identiques : `tab` est un tableau, dont l'adresse est fixe et connue à la compilation ; `p` est une variable pointeur, qui peut être réaffectée pour pointer ailleurs. `sizeof(tab)` donne la taille totale du tableau, alors que `sizeof(p)` donne la taille d'un pointeur (typiquement 8 octets sur une architecture 64 bits), quel que soit le type pointé.

6. Pointeurs de fonction (aperçu)

```
int addition(int a, int b) { return a + b; }
int soustraction(int a, int b) { return a - b; }

int (*operation)(int, int) = addition;
printf("%d\n", operation(2, 3)); // 5

operation = soustraction;
printf("%d\n", operation(5, 2)); // 3
```

Un pointeur de fonction stocke l'adresse d'une fonction, que l'on peut ensuite appeler indirectement, ou transmettre en paramètre à une autre fonction. Ce mécanisme est utilisé pour construire des *callbacks* (fonctions passées en paramètre pour être appelées plus tard, par exemple par une bibliothèque de tri générique) et des tables de dispatch (tableaux de pointeurs de fonction, indexés par exemple par un code d'opération).

Ce même mécanisme, extrêmement utile pour concevoir du code flexible, est aussi exploité par certaines techniques d'attaque de type détournement de flot de contrôle : si un attaquant parvient, via un débordement de tampon, à écraser un pointeur de fonction (ou une adresse de retour sur la pile) avec une valeur de son choix, il peut potentiellement rediriger l'exécution du programme vers un code arbitraire. C'est l'une des raisons pour lesquelles la sécurisation de la pile (protections *stack canaries*, ASLR, etc., abordées dans les cours de sécurité applicative) est aujourd'hui systématique dans les compilateurs modernes.

À retenir

- Toujours vérifier le retour de `malloc/calloc/realloc` avant d'utiliser le pointeur obtenu.
- Toute allocation dynamique doit avoir une libération correspondante, une seule fois — ni oubliée (fuite mémoire), ni dupliquée (double free).
- Mettre un pointeur à `NULL` après `free` limite fortement les risques de use-after-free accidentel, car déréférencer `NULL` échoue immédiatement et de façon détectable, plutôt que de corrompre silencieusement la mémoire.
- L'arithmétique de pointeur se fait en unités du type pointé ; un tableau « dégrade » en pointeur vers son premier élément dès qu'il est manipulé comme tel.
- Ne jamais renvoyer l'adresse d'une variable locale : sa durée de vie se termine avec le bloc qui la contient.

Chapitre 7 — Structures (`struct`) et fichiers

1. Définir une structure

```
struct Utilisateur {
    char nom[50];
    int age;
    float solde;
};

struct Utilisateur u1 = {"Zakaria", 30, 1500.0f};
printf("%s a %d ans\n", u1.nom, u1.age);
```

Une structure regroupe des données hétérogènes (de types potentiellement différents) sous un seul type nommé, accessible via l'opérateur `.` pour chaque membre. C'est la brique de base pour modéliser des objets métier cohérents (un utilisateur, un paquet réseau, un enregistrement de journal, un en-tête de fichier) plutôt que de manipuler des variables isolées dont la cohérence logique reposerait uniquement sur la discipline du programmeur.

Disposition mémoire et alignement

Les membres d'une structure sont stockés en mémoire dans l'ordre de leur déclaration, mais le compilateur peut insérer des octets de remplissage (*padding*) entre certains membres, afin de respecter les contraintes d'alignement de l'architecture cible (par exemple, un `int` doit souvent commencer à une adresse multiple de 4). En conséquence, `sizeof(struct Utilisateur)` n'est pas nécessairement égale à la somme des `sizeof` de chaque membre.

```
struct Exemple {
    char a;    // 1 octet
    int b;     // 4 octets, mais alignement à 4 : 3 octets de padding insérés avant b
    char c;    // 1 octet
};
printf("%zu\n", sizeof(struct Exemple)); // probablement 12, pas 6
```

Cette notion, souvent négligée, est essentielle dès qu'on sérialise une structure (l'écrire telle quelle dans un fichier ou sur le réseau, voir plus bas) : le padding fait partie du contenu binaire écrit, et diffère potentiellement d'une architecture ou d'un compilateur à l'autre.

2. Structures et pointeurs

```
struct Utilisateur *pu = &u1;
printf("%s\n", pu->nom);    // équivalent à (*pu).nom
```

L'opérateur `->` déréférence un pointeur vers une structure puis accède directement à l'un de ses membres, en une seule opération plus lisible que `(*pu).nom`. C'est l'un des opérateurs les plus fréquents dans du code C réel, en particulier dans les structures de données chaînées (listes, arbres), le code de systèmes d'exploitation, ou l'analyse de protocoles réseau, où l'on manipule constamment des pointeurs vers des structures plutôt que des structures directement.

Passage d'une structure en paramètre : par valeur ou par adresse

```
void afficher_par_valeur(struct Utilisateur u) {
    // reçoit une COPIE complète de la structure : coûteux si elle est volumineuse
    printf("%s\n", u.nom);
}

void afficher_par_adresse(const struct Utilisateur *u) {
    // reçoit seulement une adresse (8 octets typiquement) ; const interdit toute modif
    printf("%s\n", u->nom);
}
```

Passer une structure « par valeur » à une fonction en copie l'intégralité de son contenu, ce qui peut être coûteux en performance pour de grandes structures, et surprenant si le programmeur s'attend implicitement à ce que la fonction puisse la modifier (elle ne modifiera que sa copie locale, comme vu au chapitre 4 pour les types de base). La convention la plus courante en C consiste donc à transmettre un pointeur vers la structure, en le qualifiant de `const` lorsque la fonction ne doit pas la modifier.

3. typedef et alias de type

```
typedef struct {
    int x;
    int y;
} Point;

Point p = {3, 4};
```

`typedef` crée un alias de type, qui permet ici d'utiliser directement `Point` au lieu de `struct Point` (ou, dans cet exemple, de se passer entièrement d'un nom de structure explicite, puisque la structure est anonyme et n'est nommée qu'à travers son alias `Point`). Cette pratique est très répandue en C pour alléger l'écriture, mais elle a un revers : elle peut masquer la nature exacte d'un type (structure, pointeur, type de base) à la lecture, ce qui peut compliquer l'audit d'un code inconnu si les conventions de nommage ne sont pas suivies avec rigueur (par exemple, suffixer par `_t` les types issus d'un `typedef`, comme le fait la bibliothèque standard avec `size_t`).

4. Structures de données dynamiques : liste chaînée (introduction)


```

typedef struct Noeud {
    int valeur;
    struct Noeud *suivant;
} Noeud;

Noeud* creer_noeud(int v) {
    Noeud *n = malloc(sizeof(Noeud));
    if (n == NULL) return NULL;
    n->valeur = v;
    n->suivant = NULL;
    return n;
}

```

Cette structure combine tout ce qui a été vu précédemment : allocation dynamique (chapitre 6), pointeurs (chapitre 6), et structures. Une liste chaînée se construit en reliant des nœuds entre eux via le champ `suivant`, chaque nœud étant alloué indépendamment sur le tas.

Libération d'une liste chaînée : parcourir avant de détruire

```

void liberer_liste(Noeud *tete) {
    Noeud *courant = tete;
    while (courant != NULL) {
        Noeud *suivant = courant->suivant; // sauvegarder AVANT de libérer courant
        free(courant);
        courant = suivant;
    }
}

```

Libérer chaque nœud d'une liste chaînée illustre un piège classique déjà évoqué au chapitre 6 : il faut impérativement sauvegarder le pointeur vers le nœud suivant *avant* d'appeler `free` sur le nœud courant, car une fois libéré, lire `courant->suivant` constituerait un accès à une mémoire déjà libérée (*use-after-free*). Cette structure de données est le premier exemple concret, dans ce cours, où l'oubli d'une libération correcte peut se propager en fuite mémoire à chaque insertion, si l'ensemble de la liste n'est jamais parcouru et libéré en fin de programme.

5. Manipulation de fichiers

```
#include <stdio.h>

FILE *f = fopen("donnees.txt", "w");
if (f == NULL) {
    perror("Erreur d'ouverture");
    return 1;
}
fprintf(f, "Bonjour %s\n", "Zakaria");
fclose(f);

FILE *lecture = fopen("donnees.txt", "r");
if (lecture == NULL) {
    perror("Erreur d'ouverture");
    return 1;
}
char ligne[100];
while (fgets(ligne, sizeof(ligne), lecture) != NULL) {
    printf("%s", ligne);
}
fclose(lecture);
```

Le type `FILE` (défini dans `<stdio.h>`) représente un flux ouvert vers un fichier ; toutes les opérations de lecture/écriture passent par un pointeur `FILE *`, jamais par le fichier directement. Modes d'ouverture courants : `"r"` (lecture seule, échoue si le fichier n'existe pas), `"w"` (écriture, crée le fichier ou **écrase** son contenu existant sans avertissement), `"a"` (ajout à la fin du fichier), et leurs variantes binaires `"rb"/"wb"/"ab"` (qui, sur certains systèmes comme Windows, évitent une conversion automatique des fins de ligne appliquée en mode texte).

Point de vigilance sécurité : erreurs, tailles, chemins

- **Toujours vérifier la valeur de retour de `fopen`** : elle vaut `NULL` en cas d'échec (fichier inexistant, droits insuffisants, disque plein), et poursuivre l'exécution avec un pointeur `NULL` provoquerait un déréférencement invalide dès le premier appel à `fprintf/fgets`.
- **Borner systématiquement la taille lue** : `fgets(ligne, sizeof(ligne), f)` prend explicitement la taille du tampon de destination et s'arrête avant de la dépasser. La fonction historique `gets`, qui ne prenait aucune limite de taille en paramètre, a été supprimée du standard C11 précisément parce qu'elle rendait tout débordement de tampon quasiment inévitable dès qu'une entrée dépassait la taille prévue.
- **Valider tout chemin de fichier construit à partir d'une entrée utilisateur**. Concaténer directement une entrée utilisateur à un chemin de base (par exemple `"/donnees/" + nom_fichier_utilisateur`) expose à une vulnérabilité de type *path traversal* : un attaquant qui fournit un nom contenant `../..etc/passwd` (ou équivalent) peut potentiellement accéder à des fichiers situés en dehors du répertoire prévu. La validation doit rejeter ou neutraliser les séquences de remontée de répertoire (`..`) avant toute utilisation du chemin.

```
#include <string.h>

int chemin_est_sur(const char *nom_fichier) {
    // Vérification simplifiée à but pédagogique : rejette toute présence de ".."
    return strstr(nom_fichier, "..") == NULL;
}
```

Cette vérification reste volontairement simplifiée à des fins pédagogiques ; une validation robuste en environnement réel s'appuie généralement sur des fonctions dédiées de normalisation de chemin fournies par le système d'exploitation, plutôt que sur une recherche de sous-chaîne, insuffisante face à certains contournements (chemins absolus, liens symboliques, encodages alternatifs).

6. Lecture/écriture binaire

```
struct Utilisateur u = {"Zakaria", 30, 1500.0f};

FILE *f = fopen("utilisateur.bin", "wb");
if (f != NULL) {
    fwrite(&u, sizeof(struct Utilisateur), 1, f);
    fclose(f);
}

struct Utilisateur relue;
FILE *g = fopen("utilisateur.bin", "rb");
if (g != NULL) {
    size_t n = fread(&relue, sizeof(struct Utilisateur), 1, g);
    if (n == 1) {
        printf("%s, %d ans\n", relue.nom, relue.age);
    }
    fclose(g);
}
```

`fwrite/fread` écrivent et lisent des blocs bruts d'octets, sans aucune transformation ni interprétation : c'est ce qui les rend rapides, mais aussi rigides. Vérifier la valeur de retour de `fread` (le nombre d'éléments effectivement lus, qui peut être inférieur à ce qui était demandé si le fichier est plus court que prévu, par exemple corrompu ou tronqué) est indispensable avant d'utiliser les données lues.

Cette approche est utile pour sérialiser des structures de données, mais elle reste sensible :

- aux différences d'**alignement mémoire** entre systèmes ou compilateurs (le padding évoqué en section 1 fait partie des octets écrits, et peut varier) ;
- aux différences d'**endianness** (ordre des octets d'un nombre en mémoire : *little-endian* sur la plupart des architectures grand public, *big-endian* sur d'autres) entre la machine qui écrit le fichier et celle qui le relit.

Un fichier binaire écrit avec `fwrite` sur une architecture n'est donc pas garanti portable tel quel vers une architecture différente — un point repris en *Cryptographie* (encodage explicite des données avant chiffrement, souvent via un format indépendant de la plateforme) et en *Technologie Blockchain*

(sérialisation déterministe des transactions, un prérequis pour que différents nœuds du réseau calculent exactement le même résultat).

À retenir

- `struct` modélise des données composites cohérentes ; combinée à des pointeurs et à l'allocation dynamique, elle permet de construire des structures de données dynamiques (listes chaînées, arbres, tables).
- Le compilateur peut insérer du padding dans une structure pour respecter les contraintes d'alignement : `sizeof` d'une structure n'est pas toujours la somme des `sizeof` de ses membres.
- La gestion de fichiers en C impose une vérification systématique des erreurs (`fopen`, `fread`, `fwrite`) et une fermeture explicite (`fclose`) de chaque flux ouvert.
- Toujours borner la taille des données lues (`fgets` avec taille explicite, jamais `gets`) et valider tout chemin de fichier dérivé d'une entrée utilisateur.
- Ce chapitre clôt le socle du langage : les travaux pratiques suivants consolident ces notions sur des cas pratiques complets, combinant structures, pointeurs, tableaux et fichiers.